

packages/graph-core/src/analyzer/llm-analyzer.test.ts

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:42:46.015Z

Overview

A new test file `packages/graph-core/src/analyzer/llm-analyzer.test.ts` (lines R1-248) has been added. It exercises the LLM Analyzer utilities: prompt builders and response parsers.

Key changes

- **Imports added** (R1-R6):

```
``ts import { describe, it, expect } from "vitest"; import { buildFileAnalysisPrompt, buildProjectSummaryPrompt, parseFileAnalysisResponse, parseProjectSummaryResponse, } from "../llm-analyzer.js";
```

- `buildFileAnalysisPrompt` checks that the generated prompt contains the file path, content, project context, and JSON markers (`fileSummary`, `JSON`).
- `buildProjectSummaryPrompt` verifies inclusion of a file list, optional sample file contents, and required metadata keys (`description`, `frameworks`, `layers`).
- **Parser tests**:
 - `parseFileAnalysisResponse` is exercised against valid JSON, markdown-wrapped JSON, missing fields, and invalid inputs, ensuring defaults for `complexity` and graceful handling of optional data.
 - `parseProjectSummaryResponse` tests valid JSON, markdown fences, missing fields, and invalid JSON, confirming defaults for `frameworks` and `layers`.

Impact

- **Correctness**: The tests assert that prompt construction and JSON parsing behave as documented, catching regressions in formatting or default handling.
- **Maintainability**: Future changes to the analyzer functions will be immediately flagged by these tests, reducing silent failures.
- **Observability**: Test failures will surface detailed expectations (e.g., missing `fileSummary` or incorrect `complexity`), aiding debugging.

Risks & follow-ups

- **Test flakiness**: If the analyzer outputs change (e.g., new metadata keys), the tests will fail; review and update expectations accordingly.
- **Snapshot drift**: The tests rely on string containment; ensure that any intentional prompt format changes are reflected in the test logic.
- **Linting**: Run `npm run lint` to confirm the new file complies with the repo's style rules.
- **Coverage gaps**: Verify that all edge cases (e.g., empty file content) remain covered after future refactors.